Sprawozdanie RSAD#3 x .2020
Kody wykonanych programów z odpowiednim wyjaśnieniem działania zastosowanych dyrektyw i instrukcji.

1. **Opisz znaczenie poszczególnych bitów rejestrów konfiguracyjnych SPI i przetwornika ADC**

PRZETWORNIK ADC:

A) ADC Multiplexer Selection Register – ADMUX

Bit	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

•bity 7:6 - REFS1:0: Reference Selection Bits- bity źródła referencyjnego
Domyślnie ustawione jest napięcie zewnętrzne, które należy podłączyć (REFS1=1, REFS0=0). W naszym wypadku wybieramy napięcie zasilające dołączone do końcówki AVCC (REFS1=0, REFS0=1),

•bit 5 – ADLAR: ADC Left Adjust Result-
Bit powodujący przesunięcie danych. Określa sposób zapisu danych w rejestrach przechowujących wynik przetwarzania.

•bity 4:0 – MUX4:0: Analog Channel and Gain Selection Bits- wybór kanału wejściowego
Te bity związane są z wyborem źródła napięcia analogowego. Osiem wejść, różnorodna praca w trybie różnicowym z odpowiednimi wzmocnieniami.
W naszej konfiguracji wybieramy podstawową nastawę MUX4_0 =00000- napięcie należy dołączyć do wejścia ADC0 na porcie A.

B)ADC Control and Status Register A – ADCSRA- rejestr kontrolny

Bit	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7 – ADEN: ADC Enable
Bit odpowiedzialny za uaktywnienie przetwornika

• Bit 6 – ADSC: ADC Start Conversion
Ten bit należy ustawić, jeśli chcemy rozpocząć konwersję

• Bit 5 – ADATE: ADC Auto Trigger Enable
Ustawienie trybu pracy z automatycznym wyzwalaniem

• Bit 4 – ADIF: ADC Interrupt Flag
Bit ten informuje, że konwersja się zakończyła i możemy odczytać dane. Ten bit jednocześnie jest bitem, który wyzwala potencjalne przerwania, gdybyśmy chcieli z nich korzystać.

• Bit 3 – ADIE: ADC Interrupt Enable
Możliwość wyboru przerwania od wyniku.

• Bits 2:0 – ADPS2:0: ADC Prescaler Select Bits
Związane z preskalerem częstotliwości taktowania naszego przetwornika- wybór stopnia podziału preskalera

C)The ADC Data Register – ADCL and ADCH- bity wyniku
Przetwornik jest 10 bitowy więc wynik mamy na młodszym bajcie. Najmłodsze bity są najbardziej narażone na zakłócenia/ szumy na zasilaniu, powodują że bity ADC0 i ADC1 są najmniej “pewne”-mało wiarygodne, dlatego możemy je odciąć przesuwając te bity i korzystając tylko ze starszej części, co jest możliwe ustawiając bit ADLAR =1. Wynik tak naprawdę 8-bitowy mamy w starszym bajcie- ADCH.

ADLAR = 0

Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	–	–	ADC9	ADC8	ADCH
	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R	R	R	R	
	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

ADLAR = 1

Bit	15	14	13	12	11	10	9	8	
	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
	ADC1	ADC0	–	–	–	–	–	–	ADCL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R	R	R	R	
	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

D) Special FunctionIO Register – SFIOR- Rejestr funkcji specjalnych
Rejestr związany z timerami i komparatorem

INTERFEJS SZEREGOWY SPI:

Table 25. Port B Pins Alternate Functions

Port Pin	Alternate Functions
PB7	SCK (SPI Bus Serial Clock)
PB6	MISO (SPI Bus Master Input/Slave Output)
PB5	MOSI (SPI Bus Master Output/Slave Input)
PB4	$\overline{SS}$ (SPI Slave Select Input)
PB3	AIN1 (Analog Comparator Negative Input) OC0 (Timer/Counter0 Output Compare Match Output)
PB2	AIN0 (Analog Comparator Positive Input) INT2 (External Interrupt 2 Input)
PB1	T1 (Timer/Counter1 External Counter Input)
PB0	T0 (Timer/Counter0 External Counter Input) XCK (USART External Clock Input/Output)

PB7-SCK-linia taktująca
PB6-linia powrotna MISO-Master Input/Slave Output,
PB5- linia danych MOSI- Master Output/Slave Input, przez którą wysyłamy
PB4- Slave Select Input- linia, która wybiera nam dany układ slave przy pracy z wieloma odbiornikami

Z układem SPI łączą się 3 rejestry:
A)SPI Control Register – SPCR -Rejestr kontrolny

Bit	7	6	5	4	3	2	1	0	
	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

•Bit 7-SPIE-
Odblokowanie przerwania SPI.

•Bit 6-SPE-SPI Enable
Bit włączenia interfejsu SPI.

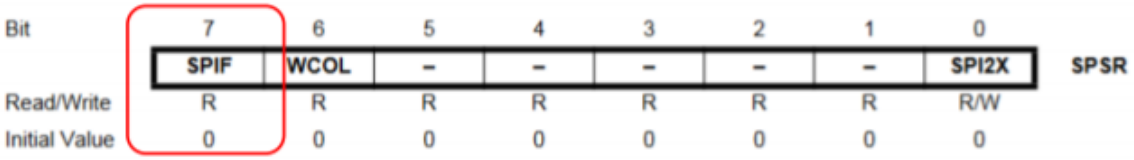
• Bit 5 – DORD: Data Order-
Bit ustawiony zmienia kolejności wysyłania danych na od najmłodszego bitu.

•Bit 4- • Bit 4 – MSTR: Master/Slave Select
W zależności od tego czy układ działa jako MASTER czy SLAVE

•Bit 3:2 – CPOL: Clock Polarity and CPHA: Clock Phase
Związane z polaryzacją i fazą sygnału zegarowego- możemy wyzwalać zboczem narastającym lub opadającym

•Bit1:0-SPR1:0 SPI Clock Rate Select 1 and 0
Związane z prędkością przesyłania układu taktującego- podział częstotliwości do taktowania (przez wielokrotność 2)

B)SPI Status Register – SPSR- Rejestr statusowy



Bit	7	6	5	4	3	2	1	0	
	SPIF	WCOL	–	–	–	–	–	SPI2X	SPSR
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

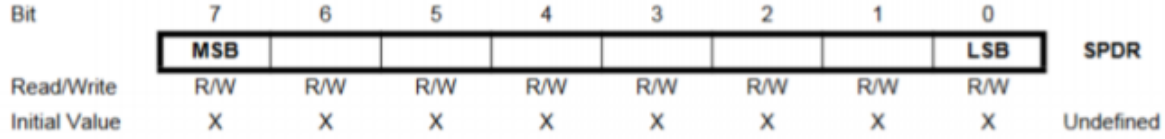
• Bit 7 – SPIF: SPI Interrupt Flag
Bit kontroli, czy dane zostały przesłane. Może być wykorzystany do aktywacji przerwania

• Bit 6 – WCOL: Write COLlision Flag
Kontrola kolizji zapisu, zakłóceń transmisji.

• Bit 5:1 – Res: Reserved Bits

• Bit 0 – SPI2X: Double SPI Speed Bit
Podwójna szybkość SPI.

C)SPI Data Register – SPDR-Rejestr danych SPI



Bit	7	6	5	4	3	2	1	0	
	MSB							LSB	SPDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	X	X	X	X	X	X	X	X	Undefined

8-bitowy rejestr danych SPI służy do odczytu i zapisu transmitowanych danych. Zapis do rejestru inicjuje transmisję. Odczytywanie rejestru powoduje odczyt buforu odbiorczego rejestru przesuwnego.

2. **Program z zajęć:**

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp 0x0030
```

```
// wektory przerwań
.org 0x0016
jmp RX_Complete// adres z którego zaczynaliśmy został zapisany
.org 0x0030
START:
//ustawienie stosu
ldi r16,LOW(RAMEND)
out SPL, r16
ldi r17,HIGH(RAMEND)
out SPH, r17

//linia PB0 jako wyjście
ldi r16, 0b00000001
out DDRB, r16//ustawia jedną linie PORTU B jako wyjście

//port PC jako wyjście
ldi r16, 0b11111111
out DDRC, r16

// ustawienie parametrów UART- prędkość
; zapisujemy rejestr UBRR
ldi r16, 25; dla 8MHz 9600
out UBRRL, r16
ldi r16, 0
out UBRRH, r16


; uaktywnienie nadajnika (Enable transmitter)
ldi r16, (1<<RXCIE)|(1<<TXEN )|(1<<RXEN)/* 1 zostaje przesunięty w lewo o
tyle pozycji TXEN ----1<<RXCIE zezwala na przerwanie od portu
szeregowego*/
out UCSRB,r16

; format ramki (Set frame format): 8data, 1stop bit
ldi r16, (1<<URSEL)|(0<<USBS)|(1<<UCSZ1)|(1<<UCSZ0)
out UCSRC,r16


// przykładowa konfiguracja przetwornika ADC
ldi R16,(1<<REFS0)|(1<<ADLAR)//ustawiamy starszy bajt z wynikiem
out ADMUX, R16

ldi R16,(1<<ADEN)|(1<<ADPS1)|(1<<ADPS0)//|(1<<ADSC)
/*aktywowanie przetwornika, szybkość taktowania (częstotliwość zegara
dzielona jest przez 8)*/
out ADCSRA, R16



; ustawienia interfejsu SPI---------------------------------
; ustawienie SCK, MOSI, SS jako wyjścia
ldi r16, (1<<DDB7)|(1<<DDB5)|(1<<DDB4)
;0b10110000
out PORTB, r16
```

```
; SPI jako MASTER
ldi r16, (1<<SPE)|(1<<MSTR)|(1<<SPR0)
out SPCR, r16
;-----------------------------------------------------------

/*
rejestr wskaźnikowy X tworzą 2 rejestry r26 i r27
muszę zapisać adres od którego chce zapisywać dane:

LDI R26,LOW(0X0060)
LDI R27,HIGH(0X0060)
*/

ldi r17,255
sei//odblokowanie globalnych przerwań
//-----------------------------------------------------------------------


MAIN:
call SPI_MasterTransmit
nop
nop
nop


jmp MAIN


//---koniec głównego programu---
// --- podprogram-----------------------------------------------------//
.org 0x0100
//-----------------------------------------------------------------------
wait_1s:
ldi  r18, 41
ldi  r19, 150
ldi  r20, 128
L1: dec  r20
brne L1
dec  r19
brne L1
dec  r18
brne L1
ret
//-----------------------------------------------------------------------


// podprogramy sprzętowego nadajnika i odbiornika:
USART_Transmit:
; Wait for empty transmit buffer
sbis UCSRA,UDRE
rjmp USART_Transmit
; Put data (r16) into buffer, sends the data:
out UDR,r16

ret
```

```
// ----------------------------------------------------
USART_Receive:
; Wait for data to be received
sbis UCSRA, RXC
rjmp USART_Receive
; Get and return received data from buffer
in r16, UDR

ret

//podprogram przerwania od RXC
RX_Complete:
in r16, UDR
out PORTA, r16
CALL TEST_ZNAKU
//call USART_Transmit
reti//ustawia flagę I- automatycznie odblokowuje przerwanie

// podprogram testowania znaku i zapisu do RAM
// $GPGGA,183730 - fragment do wyodrębnienia
//jeśli wykryje znaki $GPGGA będę mieć pewność, że dalsze znaki będą
oznaczać godzinę

TEST_ZNAKU:
inc r17 // licznik znaków- zwiększany przy każdym nowym znaku
//wartośc startowa dla r17 to 255, dlatego po inc r17 przeskoczy on na 0

cpi r17, 0 // czy szukamy znaku na pozycji 0
breq Wykryj_Dollar
cpi r17, 1 // czy szukamy znaku na pozycji 1
breq Wykryj_G
cpi r17, 2 // czy szukamy znaku na pozycji 2...
breq Wykryj_P
cpi r17, 3
breq Wykryj_G
cpi r17, 4
breq Wykryj_G
cpi r17, 5
breq Wykryj_A
cpi r17, 6
breq Wykryj_Comma
cpi r17, 7
breq Cyfra_01
cpi r17, 8
breq Cyfra_02
cpi r17, 9
breq Cyfra_03
cpi r17, 10
breq Cyfra_04
cpi r17, 11
breq Cyfra_05
cpi r17, 12
breq Cyfra_06

ret
```

```
Wykryj_Dollar:
cpi r16, '$'/*instrukcja porównania cpi pozwala porównać zawartość
rejestru ze stałą- jest to porównanie bezpośrednie - nie musimy angażować
osobnego rejestru/*
brne NIE_TEN_ZNAK//jeśli r16 nie jest równe stałej '$' skaczemy do
podprogramu NIE_TEN_ZNAK
ret//jeśli r16 jest równe stałej '$' wychodzimy z podprogramu

Wykryj_G:
cpi r16, 'G'
brne NIE_TEN_ZNAK
ret

Wykryj_P:
cpi r16, 'P'
brne NIE_TEN_ZNAK
ret

Wykryj_A:
cpi r16, 'A'
brne NIE_TEN_ZNAK
ret

Wykryj_Comma:
cpi r16, ','
brne NIE_TEN_ZNAK
ret

//start konwersji adc:-----------------------------------------------------
sbi ADCSRA, ADSC/*ustaw bit ADSC w rejestrze ADCSRA , powoduje to
wyzwolenie kolejnej konwersji ADC*/


//zapis do RAM:

Cyfra_01://-------------------------------
st X+, r16/*zapis do kolejnych komórek pamięci RAM- zapisuje pod adres
wskazany przez rejestr wskaźnikowy X*/
call USART_Transmit
ret


Cyfra_02://-------------------------------
st X+, r16
call USART_Transmit
ldi r16,':'//separator godziny
call USART_Transmit
ret


Cyfra_03://-------------------------------
st X+, r16
call USART_Transmit
ret
```

```
Cyfra_04://-------------------------------
st X+, r16
call USART_Transmit
ldi r16,':'
call USART_Transmit
ret

Cyfra_05://-------------------------------
st X+, r16
call USART_Transmit
ret


Cyfra_06://-------------------------------
st X+, r16
call USART_Transmit
// odczytaj wynik ADC
ldi r16,' '
call USART_Transmit



//-------------------------------
//odczytaj wynik ADC:
READ_ADC:
sbic ADCSRA, ADIF //jeśli ADC nie gotowy, pomiń następną instrukcje

rjmp LOAD_ADC /*jeśli się skończyła przeskakujemy do LOAD_ADC
instrukcja zostanie wykonana gdy konwersja nie jest jeszcze gotowa,
tym samym pętla ta wykonuje się, gdy oczekujemy na konwersje*/
rjmp READ_ADC



//-----------------------------------------
//odbiór:
LOAD_ADC: //podprogram ładujący wartość z rejestru ADCH do r16
in R16, ADCH //załaduj wartość z ADCH do R16
out PORTC, R16
//st X+, r16
call USART_Transmit



//następna linia 0x0a + 0x0d:
LDI R16,0X0a
call USART_Transmit
LDI R16,0X0d
call USART_Transmit

;call SPI_MasterTransmit <-testowo wyslany 1 znak

ret
```

```
//-----------------------------------------

NIE_TEN_ZNAK://kasuje licznik pozycji- zaczynamy od początku
ldi r17,255//r17 - licznik znaków
ret

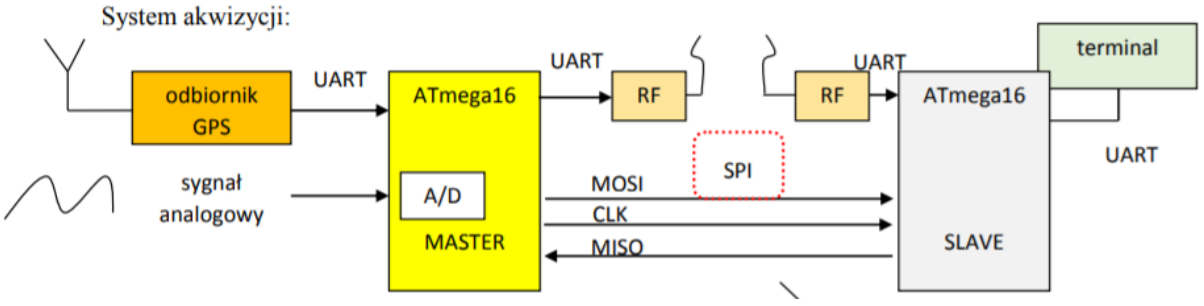


// ... podprogramy dla MASTER i SLAVE:
SPI_MasterTransmit:
ldi r16,0b11001000
; Start transmission of data (r16)
out SPDR,r16//moment przepisania do rejestru danych
Wait_Transmit:
; Wait for transmission complete
sbis SPSR,SPIF
rjmp Wait_Transmit
ret


SPI_SlaveReceive:
; Wait for reception complete
sbis SPSR,SPIF
rjmp SPI_SlaveReceive
; Read received data and return
in r16,SPDR
ret
```

Podsumowanie:

- Udało nam się napisać program odbiornika sprzętowego inicjowanego przerwaniami.
- Udało nam się co sekundę wywołać konwersję analogowo-cyfrową i zmierzyć sygnał analogowy.
- Wynik wysłaliśmy do terminala.
- Udało się wysłać komunikacją SPI do układu SLAVE



3. **Zadanie domowe 1: Napisz program dla podsystemu SLAVE odbioru danych przez SPI z obsługą przerwania od odebranego znaku i wysyłania do terminala przez UART.**

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16

.cseg
.org 0x0000
jmp START

.org 0x0014//adres przerwania w wektorze przerwań
jmp SPI_SLAVE

.org 0x0030

START:
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r17, HIGH(RAMEND)
out SPH, r17

ldi r16, 0xFF
out DDRA, r16

ldi r16, 25
out UBRRL, r16
ldi r16, 0
out UBRRH, r16

ldi r16, (1<<RXCIE) | (1<<TXEN) | (1<<RXEN)
out UCSRB,r16

ldi r16, (1<<URSEL) | (0<<USBS) | (1<<UCSZ1) | (1<<UCSZ0)
out UCSRC, r16

ldi r16, (0<<DDB7)|(0<<DDB5)|(0<<DDB4)

out PORTB, r16
//SPI jako SLAVE
ldi r16, (1<<SPIE)|(1<<SPE)|(0<<MSTR)|(1<<SPR0)
out SPCR, r16


sei //Włączenie przerwań

jmp MAIN
```

```
MAIN:
nop
nop
nop

jmp MAIN

.org 0x0100

SPI_SLAVE:
sbis SPSR, SPIF
rjmp SPI_SLAVE
in r16, SPDR
call USART_Transmit

reti

USART_Transmit:
sbis UCSRA, UDRE
rjmp USART_Transmit
out UDR, r16

ret
```

4. **Zadanie domowe 2: Napisz podprogram wysyłania do terminala wartości zapisanej w rejestrze 8-bitowym na postać heksadecymalną (znaki ASCII), czyli np. wartość 0xFF powinna zostać wyświetlana w terminalu jako FF (odpowiednie znaki ASCII).**

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp START

.org 0x0030

START:
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r17, HIGH(RAMEND)
out SPH, r17
//Ustawienie portu A jako wyjŚcie
ldi r16, 0xFF
out DDRA, r16
//Ustawienie parametrów UART'a
ldi r16, 25
out UBRRL, r16
ldi r16, 0
out UBRRH, r16
```

```
//Uaktywnienie nadajnika (Enable transmitter)
ldi r16, (1<<RXCIE) | (1<<TXEN) | (1<<RXEN)
out UCSRB,r16
//Format ramki (Set frame format): 8data, 1stop bit
ldi r16, (1<<URSEL) | (0<<USBS) | (1<<UCSZ1) | (1<<UCSZ0)
out UCSRC, r16

jmp MAIN

MAIN:
ldi r18, 0xAB //DANE 1010'1011
mov r19, r18
ldi r20, 0b11110000//maska do oddzeilenia 4 starszych bitów
and r19, r20

ror r19//przesuwamy w prawo by bity starsze były na miejscu młodszych
ror r19
ror r19
ror r19
//r19 ma starszą część danych (A)

call SEND_DATA

mov r19, r18
ldi r20, 0b00001111//maska do oddzielenia 4 młodszych  bitów
and r19, r20
//r19 ma młodszą część danych (B)

call SEND_DATA

jmp PC

.org 0x0100

SEND_DATA:
jmp DETECT_CHAR

SEND_END:
ret

//ZNAKI HEXA:
DETECT_CHAR:
ldi r16, '0'
cpi r19, 0
breq send
ldi r16, '1'
cpi r19, 1
breq send
ldi r16, '2'
cpi r19, 2
breq send
ldi r16, '3'
cpi r19, 3
breq send
```

```
ldi r16, '4'
cpi r19, 4
breq send
ldi r16, '5'
cpi r19, 5
breq send
ldi r16, '6'
cpi r19, 6
breq send
ldi r16, '7'
cpi r19, 7
breq send
ldi r16, '8'
cpi r19, 8
breq send
ldi r16, '9'
cpi r19, 9
breq send
ldi r16, 'A'
cpi r19, 10
breq send
ldi r16, 'B'
cpi r19, 11
breq send
ldi r16, 'C'
cpi r19, 12
breq send
ldi r16, 'D'
cpi r19, 13
breq send
ldi r16, 'E'
cpi r19, 14
breq send
ldi r16, 'F'
cpi r19, 15
breq send


jmp SEND_END

send:
call USART_Transmit
jmp SEND_END

USART_Transmit:
sbis UCSRA, UDRE
rjmp USART_Transmit
out UDR, r16

ret
```

5. **Zadanie 3: Napisz podprogram wysyłana zapisanych w RAM danych (czas i wartość) przez UART do terminala. Stosując komendy ANSII ESC Code sterowania znakami wyświetlanymi zaprezentuj czas w kolorze zielonym, wynik w kolorze czerwonym (można wybrać inne kolory):**
   18:37:30 FF
   18:37:31 FE
   18:37:32 CF
   ... itd
   Zastosuj znaki sterujące przejściem do następnej linii i powrotem na początek nowej linii. Podczas symulacji odczytu pamięci można "ręcznie" wpisać dane do RAM.

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp START

.org 0x0016
jmp RX_Complete

.org 0x0030

START:
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r17, HIGH(RAMEND)
out SPH, r17

ldi Xh, 0
ldi Xl, 0
ldi Yh, 0
ldi Yl, 0

//Ustawienie portu A jako wyjście
ldi r16, 0xFF
out DDRA, r16
//Ustawienie parametrów UART'a
ldi r16, 25
out UBRRL, r16
ldi r16, 0
out UBRRH, r16

//Uaktywnienie nadajnika (Enable transmitter)
ldi r16, (1<<RXCIE) | (1<<TXEN) | (1<<RXEN)
out UCSRB,r16
//Format ramki (Set frame format): 8data, 1stop bit
ldi r16, (1<<URSEL) | (0<<USBS) | (1<<UCSZ1) | (1<<UCSZ0)
out UCSRC, r16
ldi r17,255
```

```
sei

jmp MAIN

MAIN:
jmp PC

.org 0x0100

USART_Transmit:
sbis UCSRA, UDRE
rjmp USART_Transmit
out UDR, r16

ret

RX_Complete:
in r16, UDR
call TEST_ZNAKU
cpi r17, 12
breq send_data
reti

send_data:
//zielony  \e[32m
ldi r16, 92      //\
call USART_Transmit
ldi r16, 101     //e
call USART_Transmit
ldi r16, 91      //[
call USART_Transmit
ldi r16, 51      //3
call USART_Transmit
ldi r16, 50      //2
call USART_Transmit
ldi r16, 109     //m
call USART_Transmit

//czas
ldi Xl, 0

ld r16, X+
call USART_Transmit
ld r16, X+
call USART_Transmit

ldi r16, 58           //:
call USART_Transmit

ld r16, X+
call USART_Transmit
ld r16, X+
call USART_Transmit
```

```
ldi r16, 58      //:
call USART_Transmit

ld r16, X+
call USART_Transmit
ld r16, X+
call USART_Transmit

ldi r16, 32           //spacja
call USART_Transmit

//czerwony \e[31m
ldi r16, 92      //\
call USART_Transmit
ldi r16, 101     //e
call USART_Transmit
ldi r16, 91      //[
call USART_Transmit
ldi r16, 51      //3
call USART_Transmit
ldi r16, 49      //1
call USART_Transmit
ldi r16, 109     //m
call USART_Transmit

//RAM Dane
ld r18, Y+
mov r19, r18
ldi r20, 0b11110000
and r18, r20

ror r19
ror r19
ror r19
ror r19

call SEND_RAW

mov r19, r18
ldi r20, 0b00001111
and r19, r20

call SEND_RAW

//konec linii
ldi r16, 10      //LF
call USART_Transmit
reti

SEND_RAW:
jmp DETECT_CHAR
SEND_END:
ret
```

```
DETECT_CHAR:
ldi r16, '0'
cpi r19, 0
breq send
ldi r16, '1'
cpi r19, 1
breq send
ldi r16, '2'
cpi r19, 2
breq send
ldi r16, '3'
cpi r19, 3
breq send
ldi r16, '4'
cpi r19, 4
breq send
ldi r16, '5'
cpi r19, 5
breq send
ldi r16, '6'
cpi r19, 6
breq send
ldi r16, '7'
cpi r19, 7
breq send
ldi r16, '8'
cpi r19, 8
breq send
ldi r16, '9'
cpi r19, 9
breq send
ldi r16, 'A'
cpi r19, 10
breq send
ldi r16, 'B'
cpi r19, 11
breq send
ldi r16, 'C'
cpi r19, 12
breq send
ldi r16, 'D'
cpi r19, 13
breq send
ldi r16, 'E'
cpi r19, 14
breq send
ldi r16, 'F'
cpi r19, 15
breq send
jmp SEND_END

send:
call USART_Transmit
jmp SEND_END
```

```
TEST_ZNAKU:
inc r17 // licznik znaków
cpi r17, 0 // czy szukamy znaku na pozycji 0
breq Wykryj_Dollar
cpi r17, 1 // czy szukamy znaku na pozycji 1
breq Wykryj_G
cpi r17, 2 // czy szukamy znaku na pozycji 2
breq Wykryj_P
cpi r17, 3 // czy szukamy znaku na pozycji 3
breq Wykryj_G
cpi r17, 4 // czy szukamy znaku na pozycji 4
breq Wykryj_G
cpi r17, 5 // czy szukamy znaku na pozycji 5
breq Wykryj_A
cpi r17, 6 // czy szukamy znaku na pozycji 6
breq Wykryj_Comma
cpi r17, 7 // czy szukamy znaku na pozycji 7
breq Cyfra_01
cpi r17, 8
breq Cyfra_02
cpi r17, 9
breq Cyfra_03
cpi r17, 10
breq Cyfra_04
cpi r17, 11
breq Cyfra_05
cpi r17, 12
breq Cyfra_06
ret

Wykryj_Dollar:
cpi r16, '$'
brne NIE_TEN_ZNAK
ret
Wykryj_G:
cpi r16, 'G'
brne NIE_TEN_ZNAK
ret
Wykryj_P:
cpi r16, 'P'
brne NIE_TEN_ZNAK
ret
Wykryj_A:
cpi r16, 'A'
brne NIE_TEN_ZNAK
ret
Wykryj_Comma:
cpi r16, ','
brne NIE_TEN_ZNAK
ret
//zapisz do RAM
Cyfra_01:
st X+, r16
ret
```

```
Cyfra_02:
st X+, r16
ret
Cyfra_03:
st X+, r16
ret
Cyfra_04:
st X+, r16
ret
Cyfra_05:
st X+, r16
ret
Cyfra_06:
st X+, r16
ret

NIE_TEN_ZNAK:
ldi r17, 255 //r17 - licznik znaków
ret
```